BANK MANAGEMENT SYSTEM

DATABASE PROJECT REPORT

CMPE343 Database Management Systems and Programming I

Relational Database Design, DDL, DML and SQL Queries

GITHUB Repository

https://github.com/22409291/bank-system-database.git

Project Members and Their Contributions

Complete the table below with the names, student IDs and individual responsibilities of each project member.

No.	Project Member	Student ID	Contribution
1	*Moataz Elmalik*	*22409291*	*Database Design, SQL Queries, GitHub Setup*
2	*Esther Nyota*	*22014931*	*ER Diagram, Data Population*
3	*Omer Yildirim*	*22314646*	*Query Development, Testing*
4	*Yusuf Adas*	*22113354*	*Report Writing, Documentation*

1. Introduction

1.1 Project Overview

The Bank Management System is a relational database project that organizes core banking records including branches, employees, customers, accounts, loans and transactions. It demonstrates database design principles, DDL, DML and SQL queries in a realistic business scenario.

1.2 Project Objectives

The objective is to create a database that stores, updates and retrieves banking data accurately. Primary and foreign keys maintain referential integrity. The system supports data insertion, updates, soft deletion and analytical queries for reporting.

1.3 System Scope

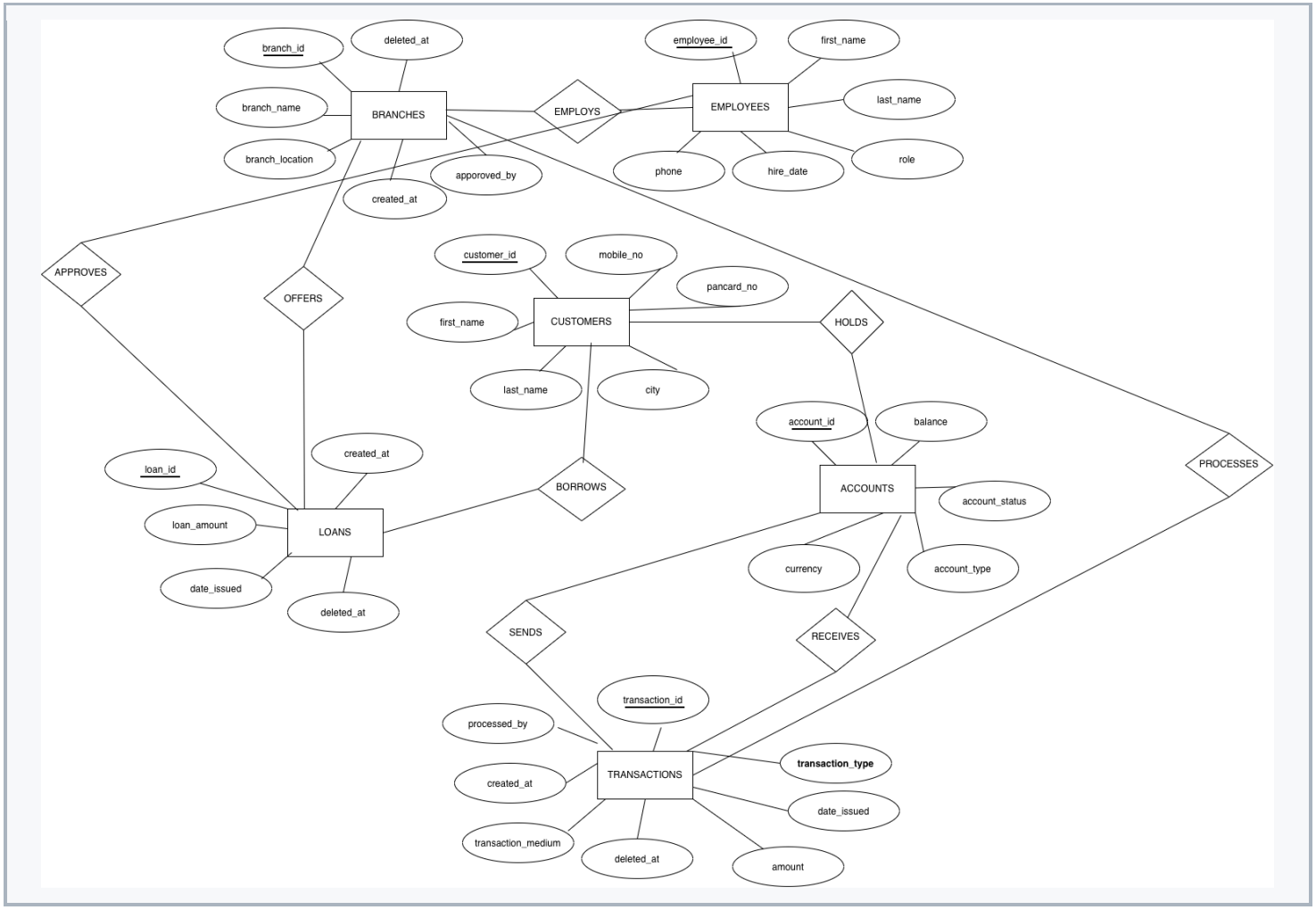
The database contains six tables: Branches, Employees, Customers, Accounts, Loans and Transactions. This simplified model demonstrates relational database operations. Authentication, interest calculation, repayment schedules and regulatory reporting are excluded.

1.4 Assumptions and Business Rules

- Each record has a unique primary key. Loan IDs are entered manually.
- One branch employs many employees; each employee belongs to one branch.
- A customer can own multiple accounts; each account belongs to one customer.
- A customer can have multiple loans; each loan connects to one customer and one branch.
- Transactions transfer between two existing accounts.
- processed_by is optional (for electronic transactions).
- approved_by in Branches can be NULL initially.
- Monetary values are stored as BIGINT.
- created_at and deleted_at support soft deletion.
- Transactions do not automatically update balances; updates are done manually via DML.

2. Database

2.1 ER Diagram



2.2 Data Definition Language Details (DDL)

2.2.1 Branches Table

```
 create table branches(
   branch_id integer unique generated by default as identity,
   branch_name varchar(20),
   branch_location varchar(20),
   approved_by int,
   created_at timestamp,
   deleted_at timestamp,
   constraint pk_branch_id primary key (branch_id),
   constraint fk_employees foreign key (approved_by) references employees(employee_id);
 );
```

2.2.2 Employees Table

```
CREATE TABLE employees (
    employee_id INTEGER GENERATED ALWAYS AS IDENTITY NOT NULL,
    branch_id INTEGER NOT NULL,
    First_Name VARCHAR(100) NOT NULL,
    Last_Name VARCHAR(100) NOT NULL,
    role VARCHAR(50) NOT NULL,
    hire_date DATE NOT NULL,
    phone VARCHAR(20),
    CONSTRAINT pk_employees PRIMARY KEY (employee_id),
    CONSTRAINT fk_branch_id FOREIGN KEY (branch_id) REFERENCES branches(branch_id)
);
```

2.2.3 Customers Table

```
 create table customers (
   customer_ID integer unique generated by default as identity,
   First_Name varchar(20),
   Last_Name varchar(20),
   City varchar(20),
   Mobile_No varchar(20),
   Pancard_No varchar(20)
   constraint pk_customer_id primary key (customer_id)
 );
```

2.2.4 Accounts Table

```
create table accounts (
      Account_ID integer unique generated by default as identity,
      Customer_ID integer,
      Balance bigint,
      Account_Status varchar(20),
      Account_Type varchar(20),
      Currency varchar(10)
      constraint pk_account_id primary key (account_id),
      constraint fk_cusomer_id foreign key (customer_id) references customers(customer_id)
);
```

2.2.5 Loans Table

```
create table loans(
  Loan_ID bigint unique NOT NULL,
  customer_id integer NOT NULL,
  Branch_ID int NOT NULL,
  Loan_Amount bigint NOT NULL,
  Date_Issued date,
  Created_At timestamp,
  deleted_at timestamp,
  constraint pk_loan_id primary key (Loan_id),
  constraint fk_customer_id foreign key (customer_id) references customers(customer_id),
  constraint fk_branch_id foreign key (branch_id) references branches(branch_id)
);
```

2.2.6 Transactions Table

```
create table transactions(
  transaction_id integer unique generated by default as identity,
  transaction_type varchar(20) NOT NULL,
  from_account_id bigint NOT NULL,
  to_account_id bigint NOT NULL,
  date_issued date,
  amount bigint NOT NULL,
  transaction_medium varchar(20),
  processed_by int,
  created_at timestamp,
  deleted_at timestamp,
  constraint pk_transaction_id primary key (transaction_id),
  constraint fk_from_account foreign key (from_account_id) references accounts(account_id),
  constraint fk_to_account foreign key (to_account_id) references accounts(account_id),
  constraint fk_employees foreign key (processed_by) references employees(employee_id)
);
```

2.3 Data Manipulation Language (DML)

INSERT Operations

```
INSERT INTO customers (First_Name, Last_Name, City, Mobile_No, Pancard_No) VALUES
('John', 'Doe', 'New York', '555-0101', 'ABCDE1234F'),
('Jane', 'Smith', 'London', '555-0102', 'FGHIJ5678K'),
('Alice', 'Brown', 'Sydney', '555-0103', 'LMNOP9012Q'),
('Bob', 'Johnson', 'Toronto', '555-0104', 'RSTUV3456W'),
('Charlie', 'Davis', 'Berlin', '555-0105', 'XYZAB7890C');
INSERT INTO branches (branch_name, branch_location, created_at) VALUES
('Downtown Hub', 'New York', '2022-03-15 09:30:00'),
('Westside Branch', 'London', '2022-06-20 14:15:00'),
('East End', 'Sydney', '2023-01-10 11:45:00'),
('North Station', 'Toronto', '2023-08-05 16:20:00'),
('Central Plaza', 'Berlin', '2024-11-12 10:00:00');
```

UPDATE Operations

```
alter table accounts
rename account_statues to account_status;
alter table accounts
add constraint chk_account_status check (account_status in ('Active', 'Inactive', 'Closed'));
```

DELETE and Soft Delete Operations

```
DELETE FROM customers
WHERE customer_id IN (1, 2, 3, 4, 5);
DELETE FROM branches;
DROP TABLE IF EXISTS branches;
```

3. Queries

3.1 Display all customers

SQL Query

```
SELECT *

FROM customers;
```

Result Screenshot

customer_id	first_name	last_name	city	mobile_no	pancard_no
1	John	Doe	New York	555-0101	ABCDE1234F
2	Jane	Smith	London	555-0102	FGHIJ5678K
3	Alice	Brown	Sydney	555-0103	LMNOP9012Q
4	Bob	Johnson	Toronto	555-0104	RSTUV3456W
5	Charlie	Davis	Berlin	555-0105	XYZAB7890C

3.2 2. Display active accounts

SQL Query

```
SELECT account_id,
       customer_id,
       balance,
       account_type,
       currency
FROM accounts
WHERE account_status = 'Active';
```

Result Screenshot

account_id	customer_id	balance	account_type	currency
2	2	25000	Savings	GBP
4	4	120000	Investment	CAD
5	5	8500	Savings	EUR
1	1	20000	Checking	USD

3.3 Display accounts with a balance greater than 30,000

SQL Query

```
SELECT account_id,
       customer_id,
       balance,
       currency
FROM accounts
WHERE balance > 30000
ORDER BY balance DESC;
```

Result Screenshot

account_id	customer_id	balance	currency
4	4	120000	CAD

3.4 Display customers and their accounts

SQL Query

```
SELECT c.customer_id, c.first_name, c.last_name, a.account_id,
       a.account_type, a.balance, a.currency
FROM customers AS c
JOIN accounts AS a
ON c.customer_id = a.customer_id;
```

Result Screenshot

customer_id	first_name	last_name	account_id	account_type	balance	currency
2	Jane	Smith	2	Savings	25000	GBP
3	Alice	Brown	3	Checking	5000	AUD
4	Bob	Johnson	4	Investment	120000	CAD
5	Charlie	Davis	5	Savings	8500	EUR
1	John	Doe	1	Checking	20000	USD

3.5 Display employees and their branches

SQL Query

```
SELECT e.employee_id, e.first_name, e.last_name, e.role, b.branch_name,
       b.branch_location
FROM employees AS e
JOIN branches AS b
ON e.branch_id = b.branch_id;
```

Result Screenshot

employee_id	first_name	last_name	role	branch_name	branch_locati
1	Michael	Scott	Manager	Downtown Hub	New York
2	Dwight	Schrute	Assistant	Westside Branch	London
3	Jim	Halpert	Teller	East End	Sydney
4	Pam	Beesly	Clerk	North Station	Toronto
5	Angela	Martin	Accountant	Central Plaza	Berlin

3.6 Display customers who have loans

SQL Query

```
SELECT c.customer_id, c.first_name, c.last_name l.loan_id, l.loan_amount, l.date_issued
FROM customers AS c
JOIN loans AS l
ON c.customer_id = l.customer_id;
```

Result Screenshot

customer_id	first_name	last_name	loan_id	loan_amount	date_issued
1	John	Doe	1001	50000	2025-06-01
2	Jane	Smith	1002	15000	2025-07-15
3	Alice	Brown	1003	300000	2025-08-20
4	Bob	Johnson	1004	10000	2025-09-10
5	Charlie	Davis	1005	75000	2025-10-05

3.7 Display loans with branch information

SQL Query

```
SELECT l.loan_id, l.loan_amount, l.date_issued, b.branch_name, b.branch_location
FROM loans AS l
JOIN branches AS b
ON l.branch id = b.branch id;
```

Result Screenshot

loan_id	loan_amount	date_issued	branch_name	branch_locati
1001	50000	2025-06-01	Downtown Hub	New York
1002	15000	2025-07-15	Westside Branch	London
1003	300000	2025-08-20	East End	Sydney
1004	10000	2025-09-10	North Station	Toronto
1005	75000	2025-10-05	Central Plaza	Berlin

3.8 Display transaction details with sender and receiver information

SQL Query

```
SELECT t.transaction_id, t.transaction_type, sender.first_name AS sender_first_name,
       sender.last_name AS sender_last_name, receiver.first_name AS receiver_first_name,
       receiver.last_name AS receiver_last_name, t.amount, t.transaction_medium, t.date_issued
FROM transactions AS t
JOIN accounts AS sender_account ON t.from_account_id = sender_account.account_id
JOIN customers AS sender ON sender_account.customer_id = sender.customer_id
JOIN accounts AS receiver_account ON t.to_account_id = receiver_account.account_id
JOIN customers AS receiver ON receiver_account.customer_id = receiver.customer_id;
```

Result Screenshot

transaction_i	transaction_t	sender_first_	sender_last_r	receiver_firs	receiver_last	amount	transaction_m	date_issued
1	Transfer	John	Doe	Jane	Smith	500	Online	2026-07-25
2	Transfer	Jane	Smith	Alice	Brown	200	App	2026-07-26
3	Transfer	Bob	Johnson	John	Doe	1500	Branch	2026-07-27
4	Transfer	Charlie	Davis	Bob	Johnson	300	Online	2026-07-28
5	Transfer	Alice	Brown	Charlie	Davis	75	App	2026-07-29

3.9 Display transactions processed by employees

SQL Query

```
SELECT t.transaction_id, t.transaction_type, t.amount, e.first_name,
       e.last_name, e.role
FROM transactions AS t
JOIN employees AS e ON t.processed_by = e.employee_id;
```

Result Screenshot

transaction_i	transaction_t	amount	first_name	last_name	role
1	Transfer	500	Michael	Scott	Manager
2	Transfer	200	Dwight	Schrute	Assistant
3	Transfer	1500	Jim	Halpert	Teller
4	Transfer	300	Pam	Beesly	Clerk
5	Transfer	75	Angela	Martin	Accountant

3.10 Calculate the total balance of all accounts

SQL Query

```
SELECT SUM(balance) AS total_account_balance
FROM accounts;
```

Result Screenshot

total_account
178500

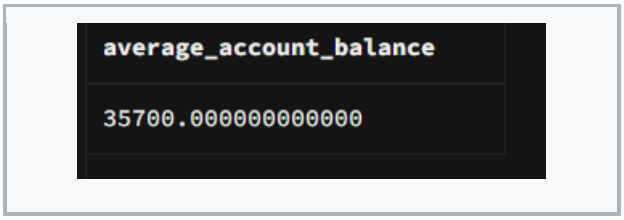
3.11 Calculate the average account balance

SQL Query

```
SELECT AVG(balance) AS average_account_balance

FROM accounts;
```

Result Screenshot



average_account_balance
35700.000000000000

3.12 Count the number of customers in each city

SQL Query

```
SELECT city, COUNT(*) AS total_customers

FROM customers

GROUP BY city

ORDER BY total_customers DESC;
```

Result Screenshot

city	total_custome
Sydney	1
New York	1
Berlin	1
London	1
Toronto	1

3.13 Count the number of employees in each branch

SQL Query

```
SELECT b.branch_name, COUNT(e.employee_id) AS total_employees
FROM branches AS b
LEFT JOIN employees AS e ON b.branch_id = e.branch_id
GROUP BY b.branch_id, b.branch_name;
```

Result Screenshot

branch_name	total_employees
North Station	1
Westside Branch	1
East End	1
Central Plaza	1
Downtown Hub	1

3.14 Calculate the total loan amount for each branch

SQL Query

```
SELECT b.branch_name, SUM(l.loan_amount) AS total_loan_amount
FROM branches AS b
JOIN loans AS l ON b.branch_id = l.branch_id
WHERE l.deleted_at IS NULL
GROUP BY b.branch_id, b.branch_name
ORDER BY total_loan_amount DESC;
```

Result Screenshot

branch_name	total_loan_amount
East End	300000
Central Plaza	75000
Downtown Hub	50000
Westside Branch	15000
North Station	10000

3.15 Display customers whose account balance is above the average

SQL Query

```
SELECT c.customer_id, c.first_name, c.last_name, a.account_id, a.balance
FROM customers AS c
JOIN accounts AS a ON c.customer_id = a.customer_id
WHERE a.balance >
(
    SELECT AVG(balance)
    FROM accounts
);
```

Result Screenshot

customer_id	first_name	last_name	account_id	balance
4	Bob	Johnson	4	120000

3.16 Display the customer with the highest account balance

SQL Query

```
SELECT c.first_name, c.last_name, a.account_id, a.balance
FROM customers AS c
JOIN accounts AS a ON c.customer_id = a.customer_id
WHERE a.balance =
(
    SELECT MAX(balance)
    FROM accounts
);
```

Result Screenshot

first_name	last_name	account_id	balance
Bob	Johnson	4	120000

3.17 Display customers who do not have a loan

SQL Query

```
SELECT c.customer_id, c.first_name, c.last_name

FROM customers AS c

LEFT JOIN loans AS l ON c.customer_id = l.customer_id

WHERE l.loan_id IS NULL;
```

Result Screenshot

customer_id	first_name	last_name
1	John	Doe
2	Jane	Smith
3	Alice	Brown
4	Bob	Johnson
5	Charlie	Davis

3.18 Display non-deleted loans

SQL Query

```
SELECT loan_id, customer_id, branch_id, loan_amount, date_issued

FROM loans

WHERE deleted_at IS NULL;
```

Result Screenshot

loan_id	customer_id	branch_id	loan_amount	date_issued
1001	1	1	50000	2025-06-01
1002	2	2	15000	2025-07-15
1003	3	3	300000	2025-08-20
1004	4	4	10000	2025-09-10
1005	5	5	75000	2025-10-05

3.19 Display total transaction amount by transaction medium

SQL Query

```
SELECT transaction_medium, COUNT(*) AS transaction_count,
       SUM(amount) AS total_amount
FROM transactions
WHERE deleted_at IS NULL
GROUP BY transaction_medium;
```

Result Screenshot

transaction_medium	transaction_count	total_amount
App	2	275
Branch	1	1500
Online	2	800

3.20 Display complete loan information

SQL Query

```
SELECT l.loan_id, c.first_name, c.last_name, b.branch_name, l.loan_amount, l.date_issued
FROM loans AS l
JOIN customers AS c
ON l.customer_id = c.customer_id
JOIN branches AS b
ON l.branch_id = b.branch_id
WHERE l.deleted_at IS NULL
ORDER BY l.loan_amount DESC;
```

Result Screenshot

loan_id	first_name	last_name	branch_name	loan_amount	date_issued
1003	Alice	Brown	East End	300000	2025-08-20
1005	Charlie	Davis	Central Plaza	75000	2025-10-05
1001	John	Doe	Downtown Hub	50000	2025-06-01
1002	Jane	Smith	Westside Branch	15000	2025-07-15
1004	Bob	Johnson	North Station	10000	2025-09-10